

软件工程概要复习

2024.6.15 by Ekaro4

- I. 简答题 (5*8分=40分)
- II. 分析题 (3*20分=60分)
- III. 平时成绩 30% 考勤和大作业
- IV. 考试成绩70%

I

蓝色是大概率会考的

红色的是极大概率会考的

给场景+分析

需求分析建模和分析(UML 图)

支持需求建模和分析的UML图

视点	图 (diagram)	说明
结构	包图 (package diagram)	从包层面描述系统的静态结构
	类图 (class diagram) ✓	从类层面描述系统的静态结构
	对象图 (object diagram)	从对象层面描述系统的静态结构
	构件图(component diagram)	描述系统中构件及其依赖关系
行为	状态图(statechart diagram)	描述状态的变迁
	活动图(activity diagram)	描述系统活动的实施
	通信图(communication diagram)	描述对象间的消息传递与协作
	顺序图(sequence diagram)✓	描述对象间的消息传递与协作
部署	部署图 (deployment diagram)	描述系统中工件在物理运行环境中的部署情况
用例	用例图 (use case diagram) ✓	从外部用户角度描述系统功能

用例 顺序 类图

一 . 软件工程基础

软件工程概念

三要素

过程+方法学+工具

软件过程模型

需求分析 概要设计 详细设计 编码实现 集成测试 确认测试

适用场景 特点

瀑布模型：只有到最后测试阶段才能看到软件

增量模型：需求确定，交付快，并行开发

迭代模型：每次迭代都是完整的开发周期，只针对部分需求

原型模型：软件原型载体媒介 支持用户参与开发 需求难导出 不易确定 持续变动

螺旋模型：引入风险驱动与风险分析

模型名称	指导思想	关注点	适合软件	管理难度
瀑布模型	提供系统性指导	与软件生命周期相一致	需求变动不大、较为明确、可预先定义的应用	易
原型模型	以原型为媒介指导用户的需求导出和评价	需求获取、导出和确认	理解需求难以表述清楚、不易导出和获取的应用	易
增量模型	快速交付和并行开发	软件详细设计、编码和测试的增量式完成	需求变动不大、较为明确、可预先定义的应用	易
迭代模型	多次迭代，每次仅针对部分明确软件需求	分多次迭代来开发软件，每次仅关注部分需求	需求变动大、难以一次性说清楚的应用	中等
螺旋模型	集成迭代模型和原型模型，引入风险分析	软件计划制定和实施，软件风险管理，基于原型的迭代式开发	开发风险大，需求难以确定的应用	难

增量和迭代的区别

软件需求：

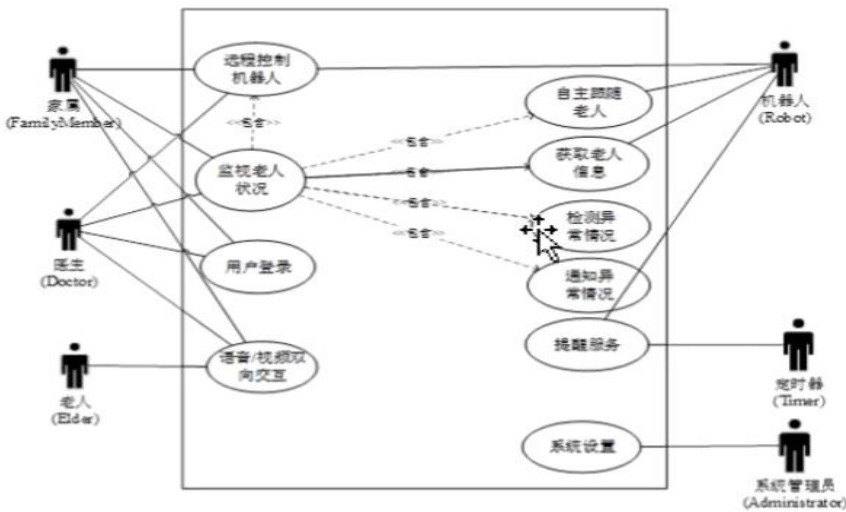
功能需求

质量需求

开发约束性需求

二．需要掌握的 UML 图（画规范即可）

用例图



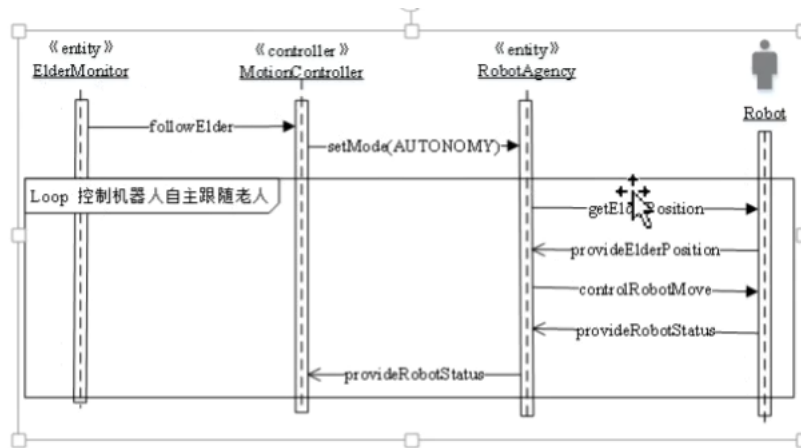
用例之间关系
包含 扩展 泛化等

包含 扩展 泛化之间的区别

用例
外部执行者
用例之间的关联

顺序图

描述对象间消息交互序列 协作
纵向时间轴 横向 对象间消息传递

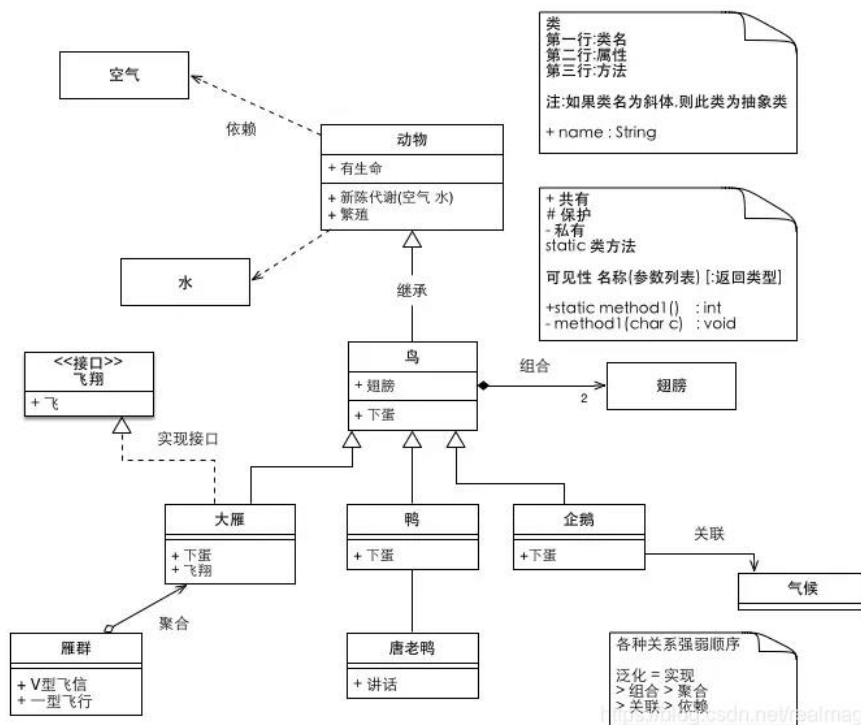


完成动作的过程

对象
时间轴
协作的消息传递过程，涉及的参数
返回的结果

类图

类： 属性+操作（顺序图）
类之间关系：实现 继承 组合 聚合 关联 依赖



类
类的属性
类之间的关系

包图？

三．概要设计

软件设计基本原则 7 个

1. 抽象与逐步求精
2. 模块化与高内聚低耦合 (why?)
3. 信息隐藏
4. 多视点与关注点分离
5. 软件重用
6. 迭代设计
7. 可追踪性

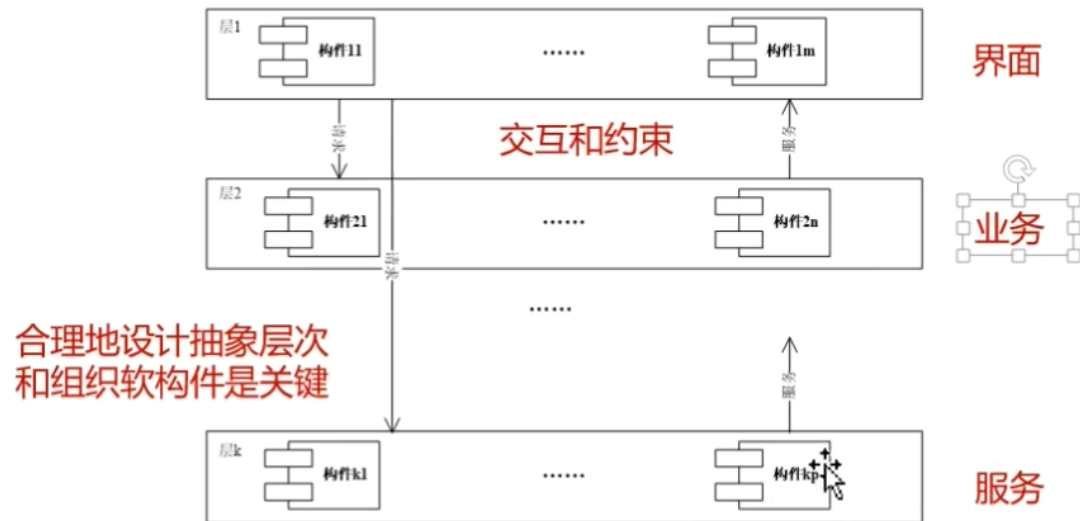
内聚：偶然性内聚->功能性内聚 低->高

耦合：非直接耦合->内容耦合

软件体系结构风格

分层

抽象程序划分层次

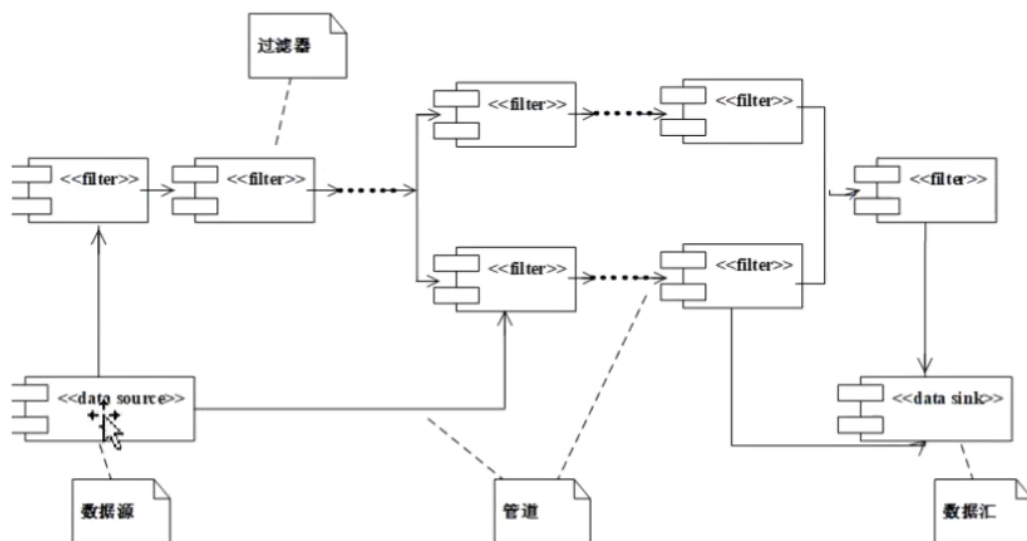


管道-过滤器

功能->一系列处理步骤，封装至一个过滤器构件，管道连接过滤器

输入：数据源 输出至：数据汇

典型：编译器



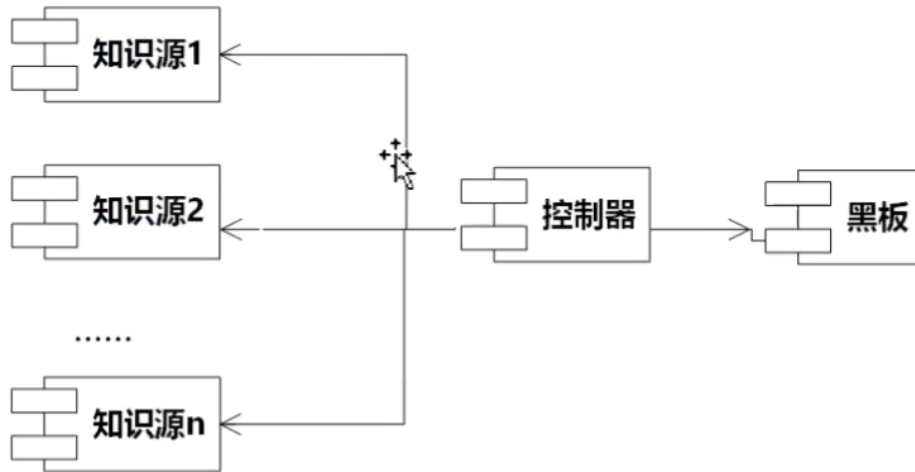
黑板

黑板+知识源+控制器

黑板：保存数据+提供读写

知识源：根据黑板的问题评价自身可应用性+负责部分求解并将结果写入黑板

控制器：监视更新状态+安排知识源的活动



MVC （model vision control）

模型 存储数据 业务逻辑处理

视图 呈现数据

控制器 从模型获取业务逻辑处理结果选择适当视图做出响应

类别	特点	典型应用
管道/过滤器风格	数据驱动的分级处理，处理流程可灵活重组，过滤器可重用	数据驱动的事务处理软件，如编译器、Web服务请求等
层次风格	分层抽象、层次间耦合度低、层次的功能可重用和可替换	绝大部分的应用软件
MVC风格 I	模型、处理和显示的职责明确，构件间的关系局部化，各个软件构件可重用	单机软件系统，Web应用软件系统
SOA风格	以服务作为基本的构件，支持异构构件之间的互操作，服务的灵活重用和组装	部署和运行在云平台上的软件系统
消息总线风格	提供统一的消息总线，支持异构构件之间的消息传递和处理	异构构件之间消息通信密集型的软件系统

四．软件测试

目的：发现缺陷

不负责解决缺陷，发现缺陷的测试是好的

单元测试

白盒测试

测试基本模块单元 内部执行过程
过程, 方法, 函数, 类

集成测试

黑盒测试

测试单元之间的接口

确认测试

测试软件功能性能, 判断是否满足需求
依据需求规格说明书
黑盒测试

其他测试

白盒测试

语句覆盖 分支覆盖 路径覆盖 基本路径覆盖

写出各种覆盖的路径和测试用例



A=2, B=0, X=2

分支覆盖

T1T2 a-c-e, A=2, B=0, X=2

F1F2 a-b-d, A=0, B=2, X=1

条件覆盖, 每个条件的真和假都要覆盖到

A>1 T; A<=1 F

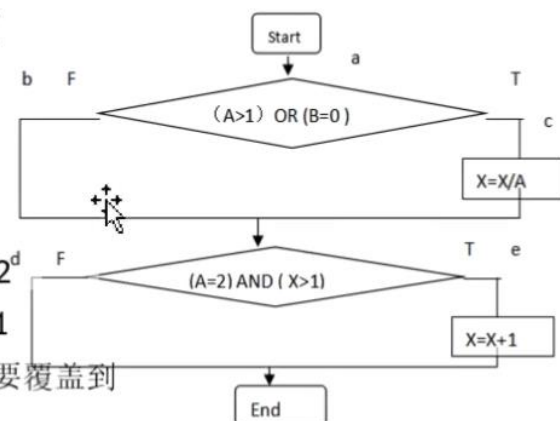
B=0 T; B!=0 F

A=2 T; A!=2 F

X>1; T; X<=1 F

路径覆盖

a-c-e, a-b-d, a-b-e a-c-d



两个测试用例 TTTT; FFFF

设计测试用例：

黑盒测试

等价分类

边界值

五．软件维护

纠正性维护

适应性维护

完善性维护

预防性维护

软件维护特点

同步性

周期长

费用高

难度大